```
from google.colab import files
uploaded = files.upload()
```

Choose Files sales_data…ata_final.csv

sales_data_final - sales_data_final.csv(text/csv) - 10820098 bytes, last modified: 4/30/2026 - 100% done

```
Saving sales_data_final - sales_data_final.csv to sales_data_final - sales_data_final.csv
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('sales_data_final - sales_data_final.csv')
print(df.shape)
df.head()
```

```
(51290, 21)
```

	order_id	order_date	ship_date	ship_mode	customer_name	segment	state	country	market	region	...	category
0	AG-2011-2040	01-01-2011	01-06-2011	Standard Class	Toby Braunhardt	Consumer	Constantine	Algeria	Africa	Africa	...	Office Supplies
1	IN-2011-47883	01-01-2011	01-08-2011	Standard Class	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania	...	Office Supplies
2	HU-2011-1220	01-01-2011	01-05-2011	Second Class	Annie Thurman	Consumer	Budapest	Hungary	EMEA	EMEA	...	Office Supplies
3	IT-2011-3647632	01-01-2011	01-05-2011	Second Class	Eugene Moren	Home Office	Stockholm	Sweden	EU	North	...	Office Supplies
4	IN-2011-47883	01-01-2011	01-08-2011	Standard Class	Joseph Holt	Consumer	New South Wales	Australia	APAC	Oceania	...	Furniture

5 rows × 21 columns

```
plt.rcParams['figure.figsize'] = (6,4)
plt.rcParams['figure.dpi'] = 100
```

```
df.info()
df.isnull().sum()

df['sales'] = df['sales'].astype(str).str.replace(',', '').astype(float)

df['order_date'] = pd.to_datetime(df['order_date'], dayfirst=True, errors='coerce')
df['ship_date']  = pd.to_datetime(df['ship_date'],  dayfirst=True, errors='coerce')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 21 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   order_id        51290 non-null  object
 1   order_date      51290 non-null  object
 2   ship_date       51290 non-null  object
 3   ship_mode       51290 non-null  object
 4   customer_name   51290 non-null  object
 5   segment         51290 non-null  object
 6   state           51290 non-null  object
 7   country         51290 non-null  object
 8   market          51290 non-null  object
 9   region          51290 non-null  object
 10  product_id      51290 non-null  object
 11  category        51290 non-null  object
 12  sub_category    51290 non-null  object
 13  product_name    51290 non-null  object
 14  sales           51290 non-null  object
 15  quantity        51290 non-null  int64
 16  discount        51290 non-null  float64
 17  profit          51290 non-null  float64
 18  shipping_cost   51290 non-null  float64
 19  order_priority  51290 non-null  object
 20  year            51290 non-null  int64
dtypes: float64(3), int64(2), object(16)
memory usage: 8.2+ MB
```

```
##Q1.Try to find out how many unique customers that we have.
```

```
unique_customers = df['customer_name'].nunique()
print(f"Total Unique Customers: {unique_customers}")
```

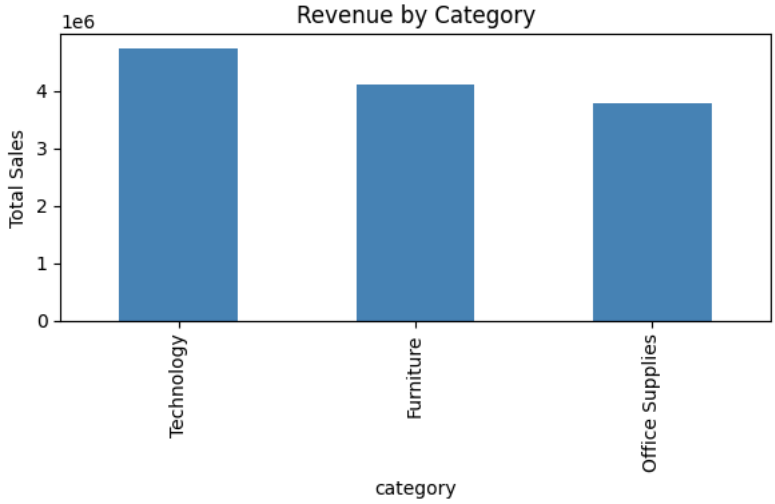
```
Total Unique Customers: 795
```

```
##Q2. What is the total sales revenue generated?

total_sales = df['sales'].sum()
print(f"Total Sales Revenue: ${total_sales:,.2f}")
```

```
Total Sales Revenue: $12,642,905.00
```

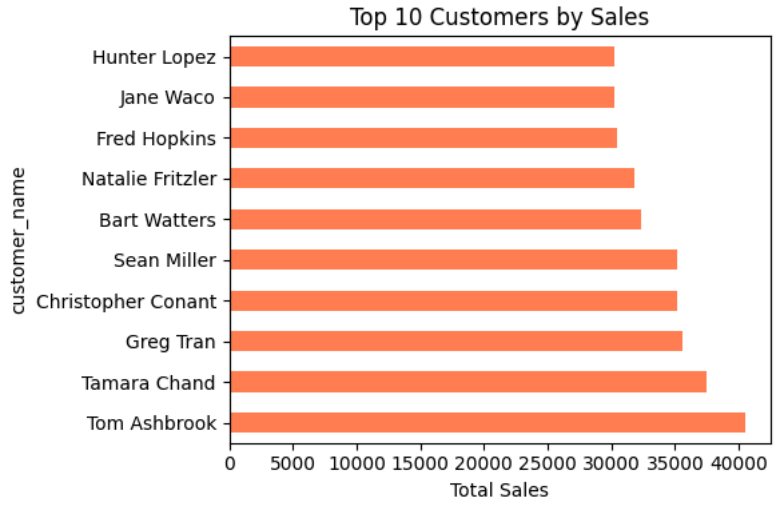
```
##Q3. Which product/category generates the highest revenue?

category_revenue = df.groupby('category')['sales'].sum().sort_values(ascending=False)
print("Revenue by Category:\n", category_revenue)

subcat_revenue = df.groupby('sub_category')['sales'].sum().sort_values(ascending=False)
print("\nTop Sub-Categories:\n", subcat_revenue.head(10))

category_revenue.plot(kind='bar', color='steelblue', title='Revenue by Category')
plt.ylabel('Total Sales')
plt.tight_layout()
plt.show()
```

```
Revenue by Category:
 category
Technology         4744691.0
Furniture          4110884.0
Office Supplies    3787330.0
Name: sales, dtype: float64

Top Sub-Categories:
 sub_category
Phones         1706874.0
Copiers        1509439.0
Chairs         1501682.0
Bookcases      1466559.0
Storage        1127124.0
Appliances     1011081.0
Machines        779071.0
Tables          757034.0
Accessories     749307.0
Binders         461952.0
Name: sales, dtype: float64
```




```
##Q4. Who are the top 10 customers by total purchase amount?

top10_customers = df.groupby('customer_name')['sales'].sum().sort_values(ascending=False).head(10)
print("Top 10 Customers:\n", top10_customers)

top10_customers.plot(kind='barh', color='coral', title='Top 10 Customers by Sales')
plt.xlabel('Total Sales')
plt.tight_layout()
plt.show()
```

```
Top 10 Customers:
 customer_name
Tom Ashbrook          40489.0
Tamara Chand          37453.0
Greg Tran             35552.0
Christopher Conant    35187.0
Sean Miller           35170.0
Bart Watters          32315.0
Natalie Fritzler      31778.0
Fred Hopkins          30404.0
Jane Waco             30288.0
Hunter Lopez          30246.0
Name: sales, dtype: float64
```

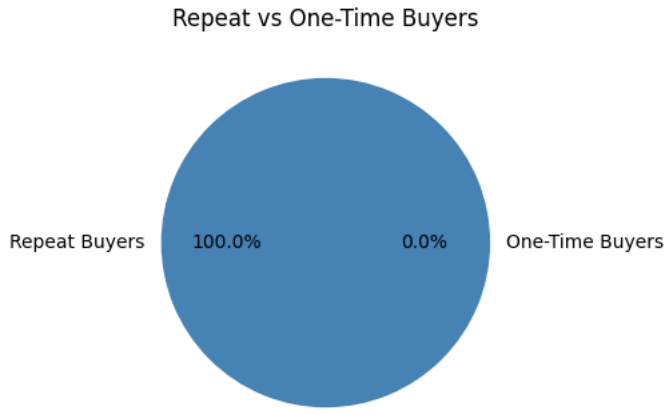



```
##Q5. Find out for which order id we have shipment duration more than 10 days.

df['ship_duration'] = (df['ship_date'] - df['order_date']).dt.days

late_shipments = df[df['ship_duration'] > 10][['order_id', 'order_date', 'ship_date', 'ship_duration']]
print(f"Orders with shipment > 10 days: {len(late_shipments)}")
print(late_shipments.head(10))
```

```
Orders with shipment > 10 days: 12305
           order_id order_date  ship_date  ship_duration
0      AG-2011-2040 2011-01-01 2011-06-01          151.0
1     IN-2011-47883 2011-01-01 2011-08-01          212.0
2      HU-2011-1220 2011-01-01 2011-05-01          120.0
3   IT-2011-3647632 2011-01-01 2011-05-01          120.0
4     IN-2011-47883 2011-01-01 2011-08-01          212.0
5     IN-2011-47883 2011-01-01 2011-08-01          212.0
6      CA-2011-1510 2011-02-01 2011-06-01          120.0
8     ID-2011-80230 2011-03-01 2011-09-01          184.0
9      IZ-2011-4680 2011-03-01 2011-07-01          122.0
10    IN-2011-65159 2011-03-01 2011-07-01          122.0
```

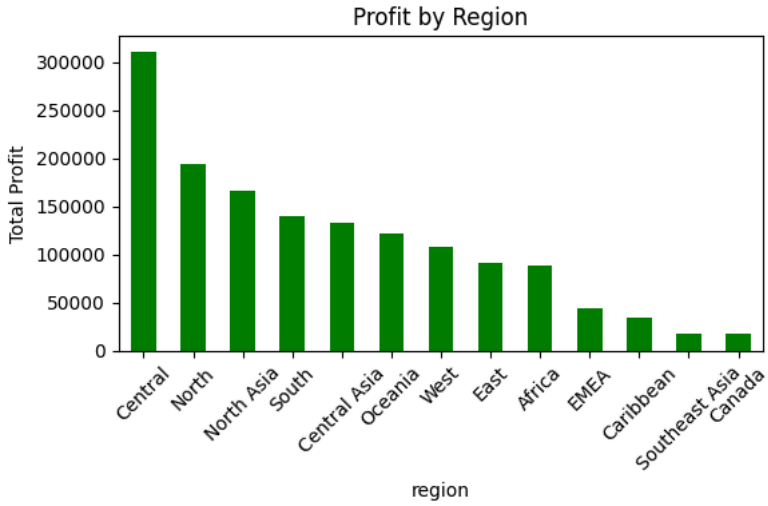
```
##Q6. How many customers are repeat buyers vs one-time buyers?

purchase_counts = df.groupby('customer_name')['order_id'].nunique()

repeat_buyers   = (purchase_counts > 1).sum()
onetime_buyers  = (purchase_counts == 1).sum()

print(f"Repeat Buyers:    {repeat_buyers}")
print(f"One-Time Buyers:  {onetime_buyers}")

# Pie chart
plt.pie([repeat_buyers, onetime_buyers],
        labels=['Repeat Buyers', 'One-Time Buyers'],
        autopct='%1.1f%%', colors=['steelblue','coral'])
plt.title('Repeat vs One-Time Buyers')
plt.show()
```

```
Repeat Buyers:    795
One-Time Buyers:  0
```



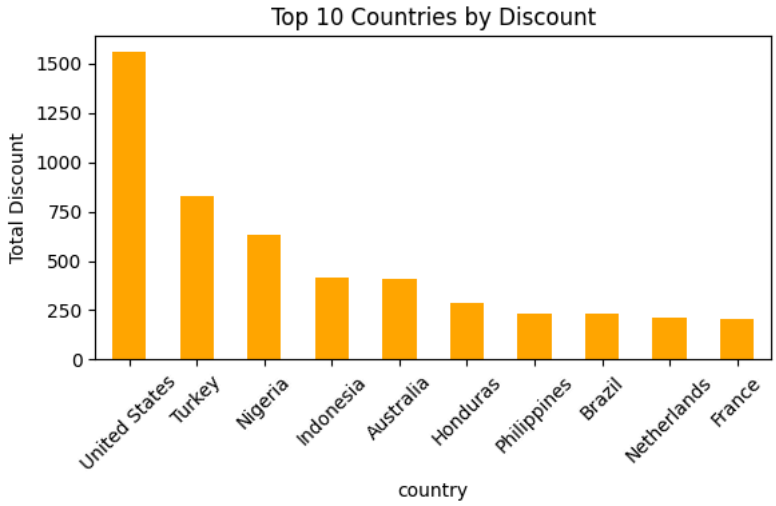

```
##Q7. Group by region and find out which region is more profitable.

region_profit = df.groupby('region')['profit'].sum().sort_values(ascending=False)
print("Profit by Region:\n", region_profit)

region_profit.plot(kind='bar', color='green', title='Profit by Region')
plt.ylabel('Total Profit')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Profit by Region:
 region
Central           311403.98164
North             194597.95252
North Asia        165578.42100
South             140355.76618
Central Asia      132480.18700
Oceania           121666.64200
West              108418.44890
East               91522.78000
Africa             88871.63100
EMEA               43897.97100
Caribbean          34571.32104
Southeast Asia     17852.32900
Canada             17817.39000
Name: profit, dtype: float64
```




```
##Q8. Try to find out overall in which country we are giving more discount.

country_discount = df.groupby('country')['discount'].sum().sort_values(ascending=False)
print("Top 10 Countries by Total Discount:\n", country_discount.head(10))

country_discount.head(10).plot(kind='bar', color='orange', title='Top 10 Countries by Discount')
plt.ylabel('Total Discount')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Top 10 Countries by Total Discount:
 country
United States    1561.090
Turkey            826.800
Nigeria           633.500
Indonesia         413.260
Australia         407.200
Honduras          290.072
Philippines       235.550
Brazil            232.822
Netherlands       209.600
France            204.350
Name: discount, dtype: float64
```



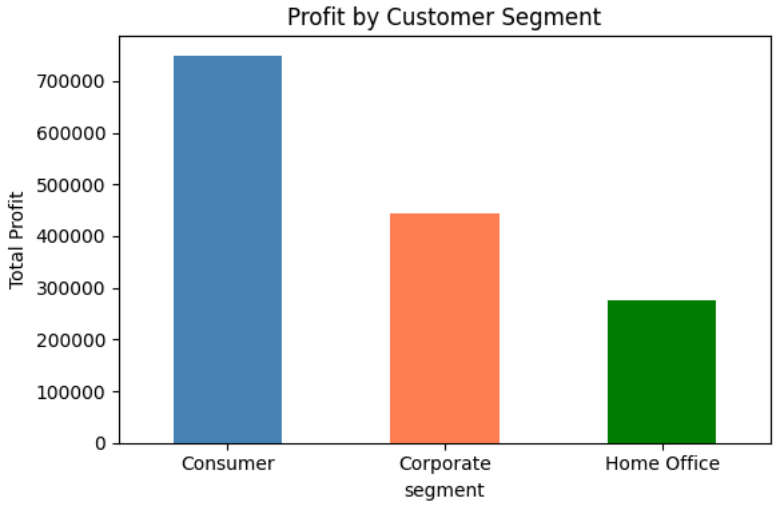

```
##Q9. Is there a relationship between quantity sold and revenue generated?

correlation = df['quantity'].corr(df['sales'])
print(f"Correlation between Quantity and Sales: {correlation:.4f}")

plt.figure(figsize=(8,5))
plt.scatter(df['quantity'], df['sales'], alpha=0.3, color='purple')
plt.xlabel('Quantity')
plt.ylabel('Sales')
plt.title('Quantity vs Sales Revenue')
plt.tight_layout()
plt.show()
```

```
Correlation between Quantity and Sales: 0.3136
```


```
##Q10.Which customer segment is more profitable find it out.

segment_profit = df.groupby('segment')['profit'].sum().sort_values(ascending=False)
print("Profit by Segment:\n", segment_profit)

segment_profit.plot(kind='bar', color=['steelblue','coral','green'],
```

```
                  title='Profit by Customer Segment')
plt.ylabel('Total Profit')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
Profit by Segment:
 segment
Consumer       749239.78206
Corporate      442785.85866
Home Office    277009.18056
Name: profit, dtype: float64
```




```
##Q11. Try to find out the 10th most loss-making product category.

subcat_profit = df.groupby('sub_category')['profit'].sum().sort_values()
print("Loss-Making Sub-Categories (sorted):\n", subcat_profit.head(15))

tenth_loss = subcat_profit.iloc[9]
print(f"\n10th Most Loss-Making Sub-Category: {subcat_profit.index[9]} with profit {tenth_loss:.2f}")
```

```
Loss-Making Sub-Categories (sorted):
 sub_category
Tables         -64083.3887
Fasteners       11525.4241
Labels          15010.5120
Supplies        22583.2631
Envelopes       29601.1163
Furnishings     46967.4255
Art             57953.9109
Machines        58867.8730
Paper           59207.6827
Binders         72449.8460
Storage        108461.4898
Accessories    129626.3062
Appliances     141680.5894
Chairs         141973.7975
Bookcases      161924.4195
Name: profit, dtype: float64

10th Most Loss-Making Sub-Category: Binders with profit 72449.85
```

```
##Q12. Try to find out 10 top products with highest margins.

product_margin = df.groupby('product_name').agg(
    total_sales=('sales', 'sum'),
    total_profit=('profit', 'sum')
).reset_index()

product_margin['margin_%'] = (product_margin['total_profit'] / product_margin['total_sales']) * 100

top10_margin = product_margin.sort_values('margin_%', ascending=False).head(10)
print("Top 10 Products by Margin:\n", top10_margin[['product_name','margin_%']])

top10_margin.set_index('product_name')['margin_%'].plot(
    kind='barh', color='teal', title='Top 10 Products by Profit Margin %')
plt.xlabel('Margin %')
plt.tight_layout()
plt.show()
```

```
Top 10 Products by Margin:
                                          product_name   margin_%
3707                                          Xerox 20  51.840000
346                                          Avery 475  50.264151
867   Canon imageCLASS MF7460 Monochrome Digital Las...  49.999749
3457            Tops Green Bar Computer Printout Paper  49.938776
3587                                         Xerox 1890  49.938776
178   Adams Telephone Message Book w/Frequently-Call...  49.875000
3233                  Southworth Structures Collection  49.863014
3734                                         Xerox 223  49.111579
3358          Strathmore #10 Envelopes, Ultimate White  49.040316
3617                                         Xerox 1918  49.012645
/tmp/ipykernel_22051/3969937595.py:16: UserWarning: Tight layout not applied. The left and right margins cannot be made larg
  plt.tight_layout()
```